

Apuntes del Curso: Computación Paralela y Distribuida

Prof. José Fiestas

15 de agosto de 2026

U1 Fundamentos de Paralelismo

El modelo de procesamiento secuencial, muestra, a lo largo de la historia, una evolución «inevitable» hacia el paralelismo

1.1. Modelo Secuencial

En el modelo secuencial suponemos que un programa está formado por una secuencia ordenada de instrucciones:

$$I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow \dots \rightarrow I_n$$

donde existe dependencia entre tareas

Modelo RAM (Random Access Machine)

Es un modelo teórico utilizado para analizar algoritmos secuenciales, donde operaciones elementales como $+$, $-$, $*$, $>$, $=$, lectura, escritura tienen un costo constante: $O(1)$

Es decir, acceder a cualquier posición de memoria cuesta lo mismo $T_{mem} = O(1)$

```
1 for (int i = 0; i < n; i++)  
2     suma += A[i];
```

Se realizan aproximadamente n operaciones, por lo que $T(n) = \Theta(n)$

Sin embargo, en una máquina real, $T(A[i]) \neq \text{constante}$, ya que el dato puede estar en: registro, caché L1/L2/L3, memoria principal. Por ello, el modelo RAM es excelente para estudiar complejidad algorítmica, pero insuficiente para explicar por sí solo el rendimiento real.

Por lo que: «Dos algoritmos con la misma complejidad asintótica pueden presentar rendimientos muy diferentes debido al acceso a memoria y a la arquitectura.»

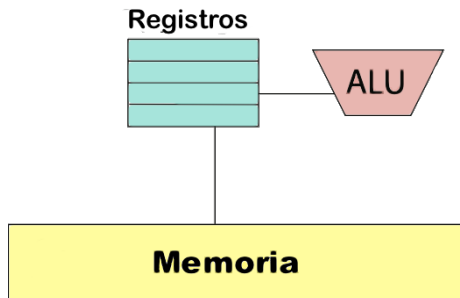
Modelo Von Neumann

La arquitectura de Von Neumann proporciona el modelo conceptual de implementación del modelo RAM. I.e. los datos y las instrucciones se almacenan en memoria (un computador pueda ejecutar distintos programas sin modificar físicamente su hardware.)

Ciclo Fetch–Decode–Execute

El funcionamiento básico puede explicarse mediante tres pasos:

- Fetch: la CPU obtiene de memoria la instrucción: memoria $\rightarrow$ CPU



- Decode: la unidad de control determina que operación debe realizarse.
- Execute: la CPU ejecuta la operación

Por ejemplo:

```

1 LOAD A
2 ADD B
3 STORE C

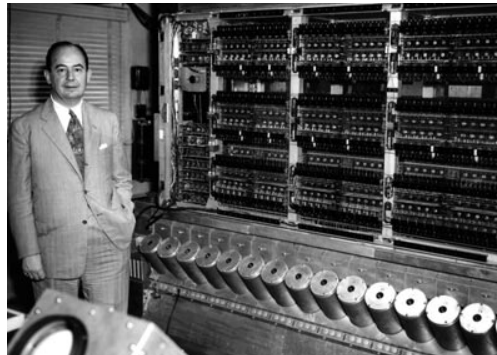
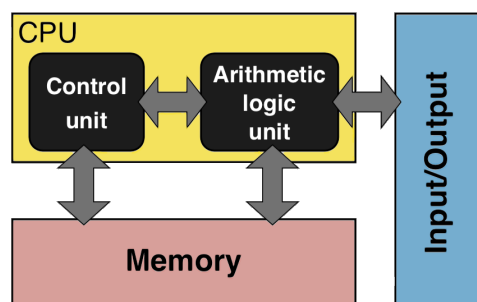
```

es decir:

Fetch → *Decode* → *Execute* → *Fetch* → ...

IAS Machine

Desarrollada en el *Institute for Advanced Study* durante las décadas de 1940 y 1950, materializó muchas de estas ideas. Hace más de 70 años se consolidó un modelo básico CPU–memoria que todavía podemos reconocer conceptualmente en los computadores actuales.



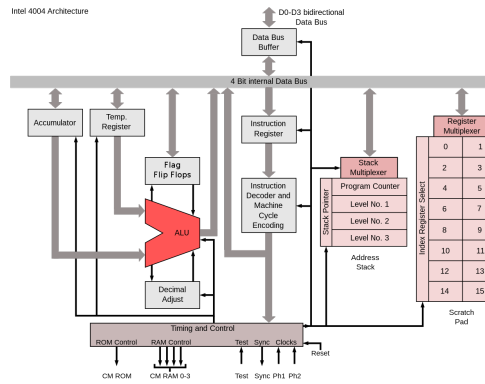
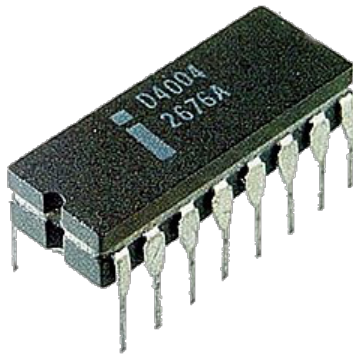
ENIAC (1945) y otras máquinas, como EDVAC (1949), procesaban instrucciones de forma secuencial. Usaban una única unidad de control y procesamiento (CPU).

De la IAS al Microprocesador

El microprocesador integra la CPU en un circuito integrado. Durante décadas, cada nueva generación permitió más transistores, mayor frecuencia, mejores cachés, pipeline más sofisticado.

El mismo programa secuencial se ejecutaba cada vez más rápido.

Intel 4004 fue el primer microprocesador (de 4 bits) en un simple chip. Lanzado en 1971. Contiene 2300 transistores y una velocidad de 740 kHz.

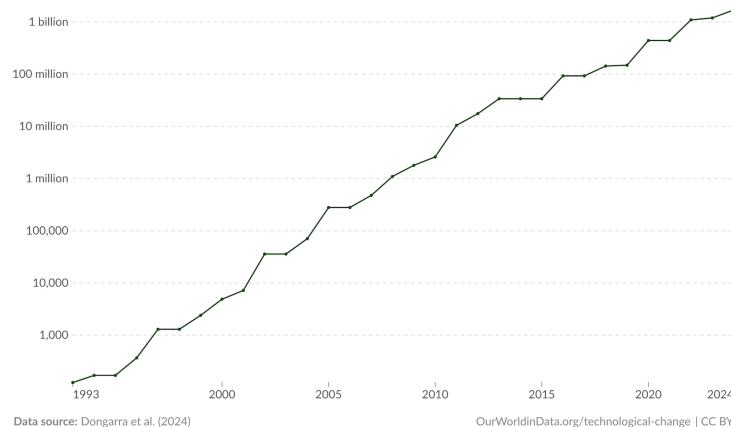


Ley de Moore:

«el número de transistores en un circuito integrado se duplica aproximadamente cada dos años», formulada por Gordon Moore en 1965, ha sido una guía para la industria de la tecnología durante décadas, impulsando la miniaturización y el aumento de la potencia de procesamiento de los dispositivos electrónicos. Actualmente, solo se mantiene vigente gracias a la programación concurrente y distribuida

Computational capacity of the fastest supercomputers

The number of floating-point operations¹ carried out per second by the fastest supercomputer in any given year. This is expressed in gigaFLOPS, equivalent to 10^9 floating-point operations per second.



Data source: Dongarra et al. (2024)

OurWorldinData.org/technological-change | CC BY

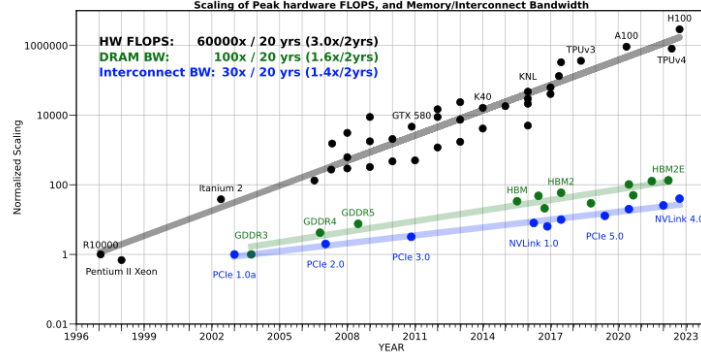
¹ Floating-point operation A floating-point operation (FLOP) is a type of computer operation. One FLOP represents a single arithmetic operation involving floating-point numbers, such as addition, subtraction, multiplication, or division.

Dongarra et al. (2024) – <https://ourworldindata.org/grapher/supercomputer-power-flops>

establece aproximadamente un crecimiento exponencial del número de transistores disponibles en los circuitos integrados

Primera limitación (Memory Wall)

Aparece el cuello de botella: **tiempo de acceso a memoria** vs **tiempo de cómputo**



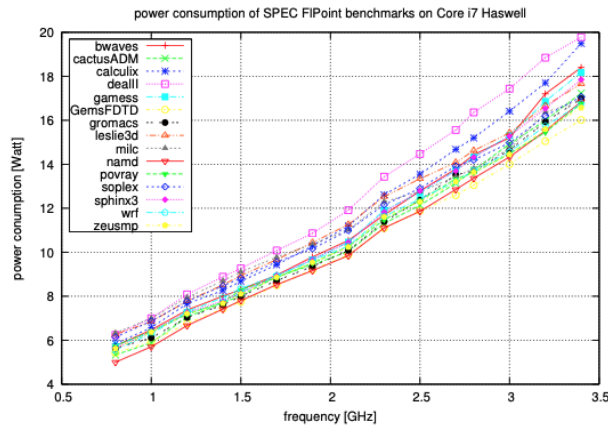
Escalabilidad de ancho de banda en distintas redes de interconexión y memoria, así como de FLOPs. Datos normalizados. De: Gholami et al. 2024 <https://arxiv.org/abs/2403.14123>

La solución fue introducir niveles intermedios: CPU $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ 3 $\rightarrow$ RAM
Cuanto más cerca de la CPU: menor capacidad, mayor velocidad, mayor costo por byte.
Cuanto más lejos: mayor capacidad, mayor latencia. Esto rompe una de las simplificaciones fundamentales del modelo RAM: $T_{acceso} \neq constante$

Segunda limitación (consumo de energía, power wall)

Incrementar la frecuencia de reloj produce más calor y consumo energético (proporcional al cubo de la frecuencia)

$$P_{dyn}(f) = \gamma \cdot f^3$$



SPEC CPU2006 floating-point benchmarks de consumo de energía en un procesador Intel Core i7 Haswell a distintas frecuencias. De: Rauber et al. 2014 <https://link.springer.com/article/10.1007/s11227-014-1236-4>

Paralelismo a nivel de instrucciones

Posibilidad de que una CPU secuencial ejecute internamente varias operaciones simultáneamente: pipeline, procesadores superescalares, múltiples unidades funcionales.

Es decir, incluso antes del multicore CPU secuencial ya explotaba internamente cierto grado de **ILP** (Instruction-Level Parallelism) Sin embargo, está limitado por dependencias de datos, saltos, dependencias entre instrucciones.

```
1 a = b + c;  
2 d = a * 2;  
3 e = d + 1;
```

Existe una cadena de dependencias.

Participación 1: Suma de números

Suma de números por **1 persona, 2 personas, 4 personas**

Obtenemos un tiempo en cada caso que no se reduce a la mitad por el tiempo necesario para la sincronización de datos: obtención de la suma acumulada. Definimos ese tiempo como el **overhead** del algoritmo, que observamos, crece con la cantidad de personas (más personas necesitan mas tiempo para sincronizar resultados parciales)

Pregunta: Defina una estrategia en la actividad de suma de numeros para reducir el overhead (tiempo de comunicación/sincronización)

Respuesta: Tenemos las siguientes alternativas

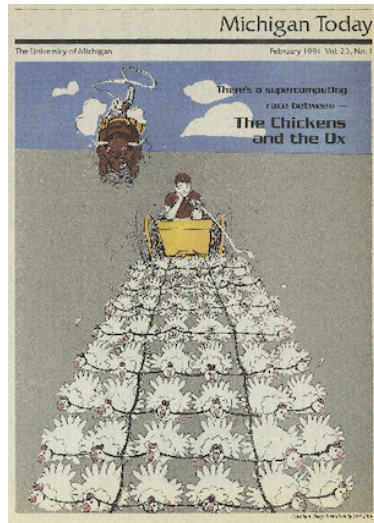
- Acumular las sumas parciales en cuanto cada persona termine el cálculo. Esta forma es no-determinística y de bajo performance (desordenada)
- Acumular las sumas parciales en forma secuencial. Esto no es óptimo ya que la comunicación es proporcional a p (personas)
- Acumular las sumas parciales por pares, esto permite que las sumas parciales se acumulen en paralelo y sucesivamente (árbol binario), considerando que una de las personas en cada etapa se encarga de la suma acumulada (para aprovechar recursos). El método deriva en una complejidad algorítmica de $O(\log p)$

Se puede optimizar también el cómputo (pipelining) pero la pregunta va dirigida al overhead, ya que esta componente del tiempo de ejecución (total) es la dominante.

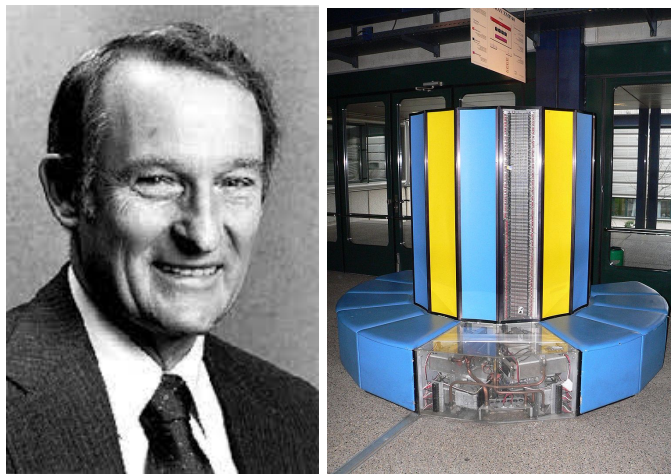
1.2. Paralelismo y arquitecturas paralelas

Seymour Cray: The Chicken and the Ox

«Para arar un campo, ¿qué preferirías usar: dos bueyes fuertes o 1024 gallinas?»
Seymour Cray (1991)



'a massively parallel computer is faster and more powerful than a conventional computer'. De: *Michigan Today*, February 1991, Vol 23, No.1



Seymour Cray (Padre de la Supercomputación) fundó CDC (Control Data Corporation) y construyó el supercomputador más rápido de la época: Cray X-MP, 800 MFlops, 4 CPUs, \$8.8 M. Compare con Apple A18, 600 GFlops, \$500

¿Es preferible tener pocos procesadores muy potentes o muchos procesadores simples? No existe una respuesta universal. Depende de la naturaleza del problema, granularidad, comunicación, sincronización, memoria, capacidad de dividir el trabajo.

Acabamos de comprobar que tener cuatro trabajadores no garantiza hacer el trabajo cuatro veces

más rápido.

Una estrategia es hacer cada procesador mas potente, mediante pipeline (distintas etapas procesan distintas instrucciones), ejecución superescalar (varias instrucciones pueden iniciarse en un mismo ciclo), SIMD (una instrucción opera sobre varios datos simultáneamente), grandes cachés. La alternativa es utilizar varias unidades de procesamiento.

Connection Machine

La Connection Machine CM-1, presentada en 1985, llevó al extremo la idea de utilizar una gran cantidad de procesadores relativamente simples.



Tamiko Thiel lideró el equipo de diseño con una topología de hipercubo de 12 dimensiones. Contiene 65536 microprocesadores (4KB RAM), logrando una velocidad de 5-10 GFLOPs. Se utilizó en modelos climáticos

La máquina podía incorporar decenas de miles de elementos de procesamiento.

Por lo tanto: **Oxen vs Chickens**

Cray : pocos recursos potentes

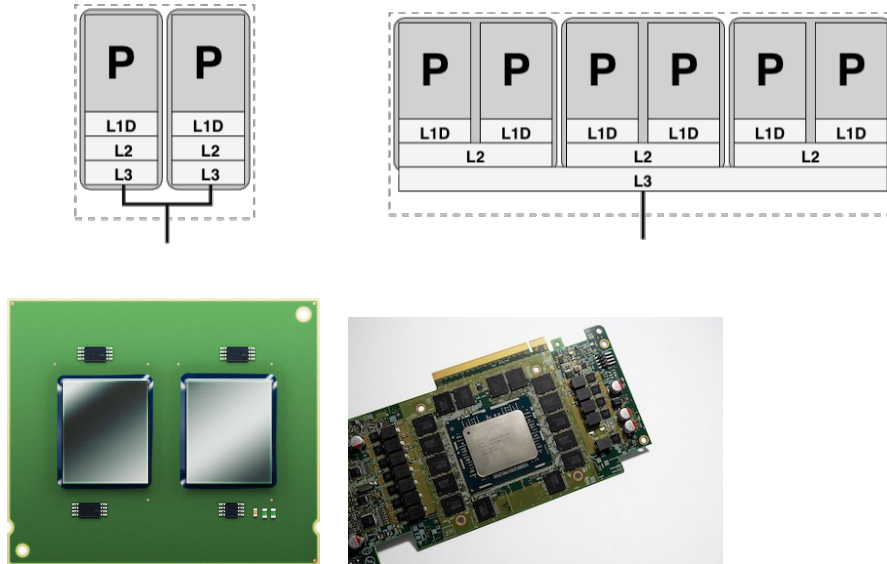
Connection Machine: muchos recursos simples

Del procesador único al multicore

Hasta principios de los años 2000, gran parte del aumento de rendimiento provenía de hacer más rápido y sofisticado un único procesador. Pero, debido al Power Wall, la solución dominante es: **más núcleos**

Términos

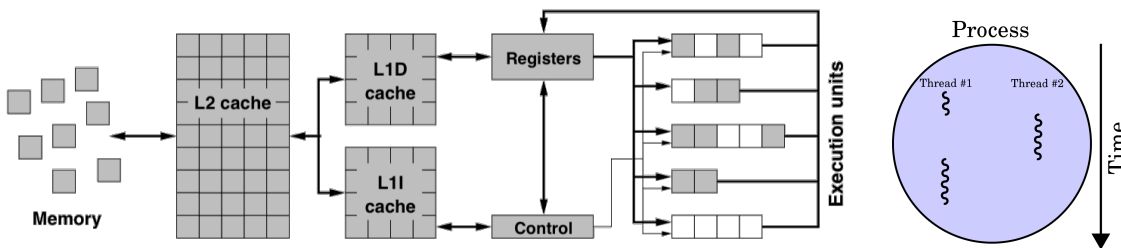
- **Procesador / CPU**: chip físico que puede contener varios núcleos.
- **Core**: unidad capaz de ejecutar instrucciones. Por ejemplo, un CPU con 8 cores
- **Thread**: un hilo es un flujo de ejecución. Un programa puede generar varios hilos (threads) que el sistema asignará a recursos de hardware. Por tanto, un **thread** es una entidad de ejecución de software, mientras que un **core** es un recurso físico.
- **Procesadores multicore**: procesadores de múltiples núcleos



E.g.: IBM POWER hasta 30 núcleos, Intel Xeon-phi hasta 72 núcleos
Arquitecturas híbridas(GPU o XeonPhi con standard CPUs)

■ Procesadores multithreading

Se trata de la implementación de multitareas como duplicados de la arquitectura del CPU (Hyperthreading o Simultaneous Multithreading). Estos son los procesadores lógicos del CPU que ejecutan tareas en paralelo



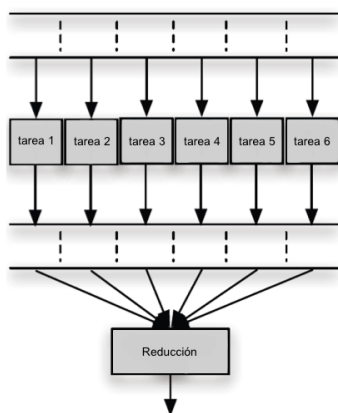
Aplica el SMT (Simultaneous Multithreading), es decir, el hardware permite que un mismo core mantenga el estado de más de un hilo. Hyper-Threading de Intel es una implementación conocida de esta idea.

- **GPUs:** prioriza throughput, gran cantidad de operaciones, muchos threads, paralelismo masivo.

Paradigmas del paralelismo (memoria compartida vs memoria distribuida)

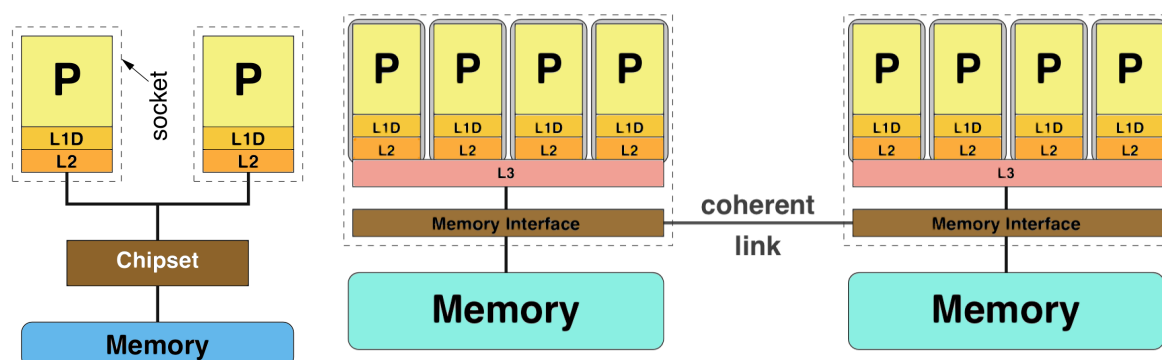
Paralelismo de la data (del resultado)

La estructura del resultado define el programa. Cada proceso solo es responsable de una tarea. La información entre los procesos no se envía, sino se lee de memoria compartida. Procesos y comunicación es implícita (OMP).



Computadoras de memoria compartida

Es un sistema en el que los CPUs comparten un mismo espacio físico (dirección). Dependiendo del acceso a memoria, pueden ser:



UMA: Uniform Memory Access, es un modelo 'plano' de acceso a memoria (latencia = ancho de banda para todos los procesadores). También llamado Simmetric Multiprocessing (SMP). E.g. procesadores multicore.

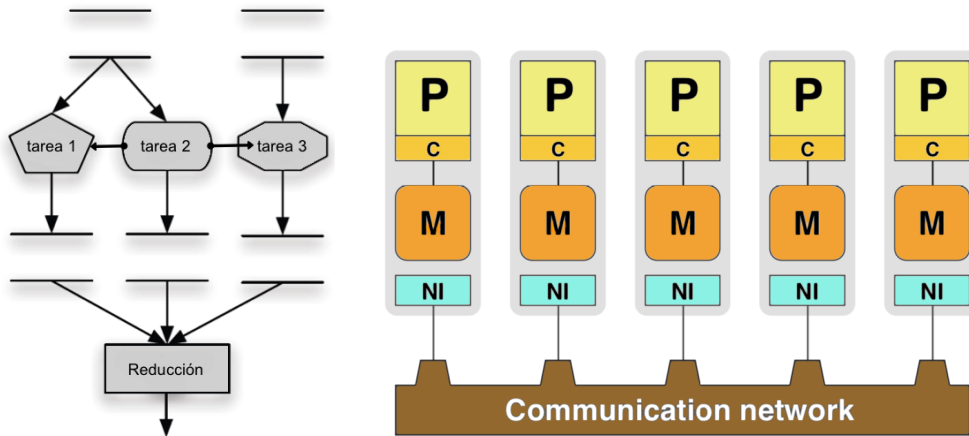
ccNUMA: Cache Coherent Non-uniform Memory Access, la memoria esta físicamente distribuida, pero una red lógica permite que funcione como memoria en un mismo espacio físico (dirección)

En ambos casos la **coherencia de Caché** permite mantener consistencia entre las cachés y la memoria en caso se realicen modificaciones por algún CPU

Paralelismo de la tarea (especialista)

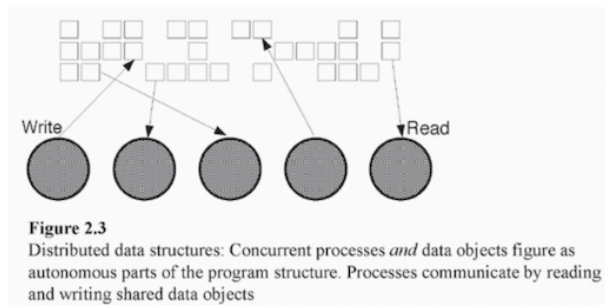
Comunicación (envío de mensajes) entre procesos. Utilizado en programación distribuída y orientada a objetos (C,C++,Fortran) No utiliza memoria compartida, sino espacio de memoria particionado en p nodos. Paralelismo es explícito (MPI). Se paraleliza la tarea (especialidad) y se agrupa a los especialistas conectados en paralelo en una red lógica

En este sistema, cada procesador está conectado a una memoria local, y ningún otro tiene acceso a esta memoria. Los procesadores se comunican a través de una red enviando a, y recibiendo mensajes de los otros procesadores.



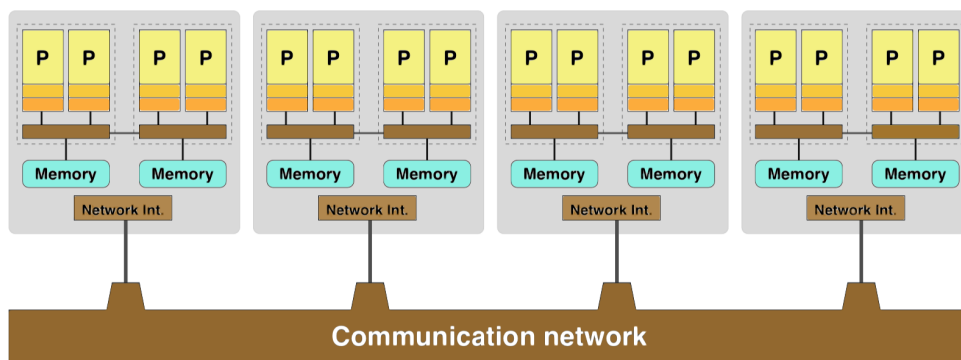
Paralelismo de la agenda (híbrido)

Caso intermedio, que mantiene la distinción entre un grupo de datos (objetos) y un grupo de procesos. Se construye enfocado en la agenda de tareas, y se asigna procesos paralelos para cumplirla.



Sistemas híbridos

En la práctica, los sistemas no son puramente distribuidos o compartidos, sino una combinación de ambos (híbridos). I.e., nodos de memoria compartida conectados a través de una red, que añaden una complejidad mayor al sistema de comunicación entre procesadores. E.g. CPU-GPU clusters



Del multicore al supercomputador

Mientras que un computador personal cuenta con varios cores, una GPU posee miles de unidades de ejecución, y un cluster muchos nodos. Un supercomputador contiene miles de nodos+CPU+GPU+red de alta velocidad

Top500 Supercomputing

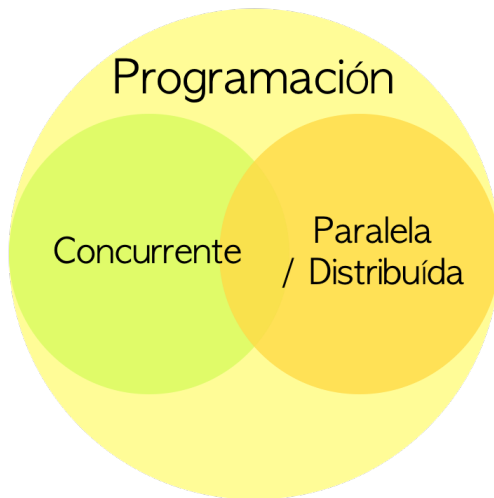


Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	LineShine - LingKun, LX2 304C 1.55GHz, LingQi, Kylin OS, Shenzhen Cloud Computing Center Co., Ltd. National Supercomputing Centre in Shenzhen (NSCS) China	13,789,440	2,198.40	2,735.82	42,220
2	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
3	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
4	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
5	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, Bull EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794

Ranking junio 2026

fuentes: <https://top500.org/lists/top500/2026/06/>

Computación Paralela



El paralelismo es una disciplina enfocada en solución **eficiente** de problemas computacionales con **infinitos recursos** para lo cual utiliza herramientas de computación **concurrente** (diseño simultáneo de tareas) y/o **distribuída** (ejecución simultánea de tareas)

- **Secuencial:**

Un único flujo $A \rightarrow B \rightarrow C$

- **Concurrente:**

Varias tareas se ejecutan en un intervalo de tiempo. E.g. ABC

pero pueden ejecutarse mediante intercalado: $A_1 \rightarrow B_1 \rightarrow A_2 \rightarrow C_1 \dots$

Por tanto, la concurrencia no exige ejecución física simultánea.

- **Paralelo:**

Dos o más tareas realmente se ejecutan simultáneamente:

$P_1 : A$

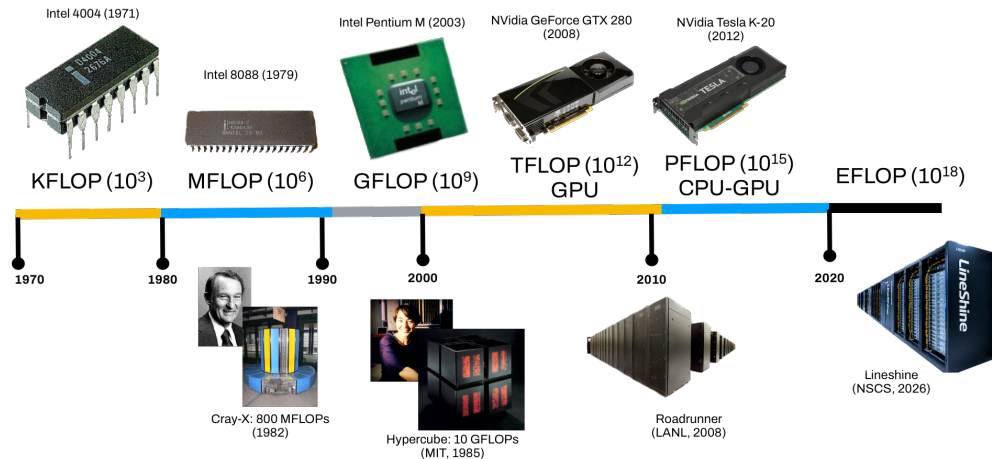
$P_2 : B$

$P_3 : C$

Esto requiere múltiples recursos físicos capaces de ejecutar trabajo simultáneamente. La concurrencia trata de estructurar múltiples tareas que pueden progresar; el paralelismo busca ejecutarlas simultáneamente para obtener rendimiento.

Evolución

- La transición del procesamiento secuencial al paralelo refleja una evolución natural de la computación motivada por la necesidad de mayor rendimiento, eficiencia energética y escalabilidad
- Refleja, en el tiempo, la preponderancia del paralelismo sobre el cómputo secuencial. El meteorólogo Lewis Fry Richardson (1922) postuló la idea de 64000 personas haciendo predicciones climáticas simultáneas y un agente de sincronización entre ellas que permitía la evolución temporal correcta. Danny Hillis (de Thinking Machines Corporation of Cambridge, Massachusetts), diseñó la Connection Machine, con 64000 procesadores, que realizó las primeras predicciones climáticas.



1.3 Modelos Escalables

Primeros Pasos hacia el Paralelismo (1960s–1980s):

Paralelismo a nivel de hardware:

- **Paralelismo a nivel de instrucción** (instruction level parallelism, ILP), segmentación o pipelining: se introdujo en procesadores como el IBM System/360. Busca aumentar la tasa en la cual las instrucciones son ejecutadas dentro de una CPU. Al dividir la ruta de ejecución en etapas discretas, la tubería permite que más de una instrucción sea ejecutada en cualquier tiempo. Esta separación puede ser comparada a una línea de ensamble, en la cual una instrucción es hecha más completa en cada etapa hasta que sale de la tubería de ejecución y es retirada del mismo
- **Paralelismo a nivel de hilos** (thread level parallelism, TLP): una tecnología utilizada para este propósito es el multiprocesamiento simétrico (SMP), donde un pequeño número de CPU comparten una visión coherente de su sistema de memoria. En este esquema, cada CPU tiene un hardware adicional para mantener una visión constantemente actualizada de la memoria. Particularmente, en el **paralelismo de datos** los procesadores vectoriales se ocupan de múltiples piezas de datos en el contexto de una instrucción (sistemas vectoriales: Cray-1, 1976). SIMD (single instruction, multiple data).

Teoría del paralelismo:

- Aparece el modelo PRAM (Parallel Random Access Machine).
- Primeras nociones de paralelismo masivo y modelo Flynn (SISD, SIMD, MISD, MIMD).

Explosión de Sistemas Paralelos (1980s–1990s)

- Supercomputadoras paralelas: Cray X-MP, Connection Machine, Thinking Machines, paralelismo a gran escala con decenas o miles de procesadores.
- Lenguajes y bibliotecas: desarrollo de MPI (1994) y PVM para programación distribuida, OpenMP (1997) para paralelismo compartido.

- Antes, se buscaba “acelerar el CPU”, ahora se comienza a diseñar algoritmos paralelos optimizados.

Transición definitiva al Paralelismo (2000s)

- La ley de Moore ya no basta. El límite térmico impide seguir aumentando la frecuencia. Los fabricantes optan por múltiples núcleos en lugar de núcleos más rápidos. Aparece el procesador multinúcleo (multicore).
- CPUs con 2, 4, 8, 64 núcleos, GPUs con miles de núcleos (CUDA de Nvidia en 2006). Lenguajes como CUDA, OpenCL para programación en GPU.

Paralelismo Masivo y Heterogéneo (2010s–presente)

- Computación heterogénea: CPUs + GPUs + FPGAs + TPUs (Google). Frameworks: TensorFlow, PyTorch, Dask, Ray.
- Cloud y Big Data. Paralelismo a nivel de clústeres y nubes. Modelos como MapReduce (Google), Spark (Apache).
- IA y Machine Learning. Redes neuronales profundas requieren miles de operaciones paralelas. Plataformas con miles de GPUs trabajan en paralelo (Ej. GPT de OpenAI).

Hay muchos problemas en la naturaleza, de ámbito científico, social y de ingeniería, que son naturalmente paralelizables (solucionables con métodos matemáticos en paralelo)

0.1. ¿Qué problemas son (naturalmente) paralelizables?

Son aquellos con las siguientes características:

- Contienen un número grande de entidades (objetos de características comunes)
- La interacción entre objetos se define por reglas simples (pero el sistema muestra un comportamiento complejo y caótico)



'A solitary ant, cannot be considered to have much of anything on his mind . . . It is only when you watch the dense mass of thousands of ants, . . . , that . . . you observe it thinking, planning, calculating. It is an intelligence, a kind of live computer, with crawling bits for its wits'. De: *Lewis Thomas, Lives of a Cell, (1974)*

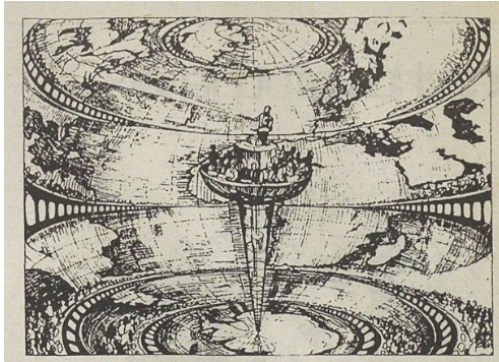
Por ejemplo, en la **economía**: un negocio beneficia solo a un individuo pero contribuye a la economía de un país

Además, estas reglas son (especialmente en Ciencia e Ingeniería):

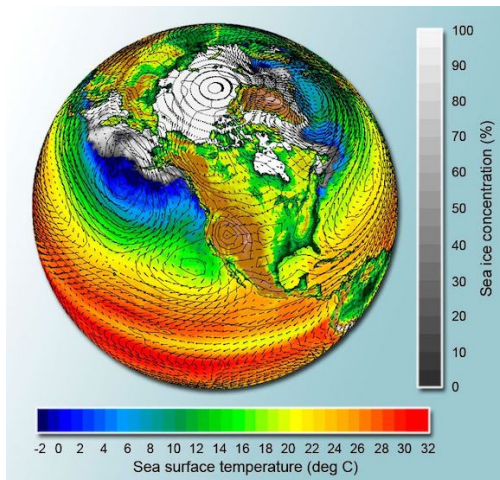
- **locales**, al poder describir fenómenos a escala pequeña
- **uniformes**, al ser aplicables a distintas localidades sin modificación

El meteorólogo Lewis Frye Richardson en su artículo *Weather Prediction by Numerical Process* (1922) describe un anfiteatro con 64000 personas calculando con ábacos y tablas de calculo, cada uno de los cuales resuelve una sola o parte de una ecuación. Todo sincronizado por una persona sobre esta estructura, rodeado de asistentes y mensajeros.

Esta idea se compara con la *Connection Machine*, diseñada por Danny Hillis, en el *Thinking Machines Corporation of Cambridge, Massachusetts* donde se reemplaza los humanos por 64000 procesadores. Sin embargo, los cuatrillones de operaciones necesarias para el procesamiento resultarían en 1000 años de calculo para un día de predicción climática.



Propuesta de L.W.Richardson, 1922: 64000 computadores humanos predicen el clima numéricamente.

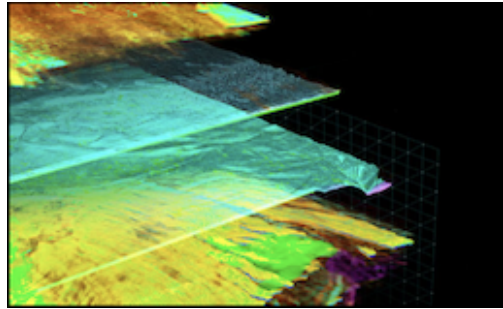


En modelos climáticos, son aplicables 5 leyes locales e uniformes: conservación de masa y momento, primera ley de la termodinámica, ecuación de estado y de humedad

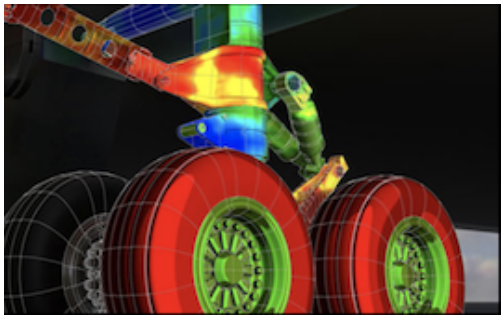
0.2. Otras aplicaciones



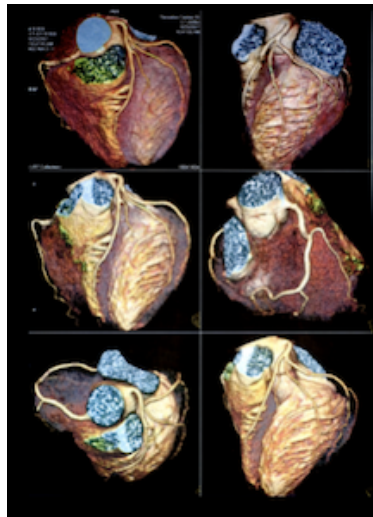
(a) Sistemas MultiGPU son usados para procesamiento gráfico de alta performance y en paralelo



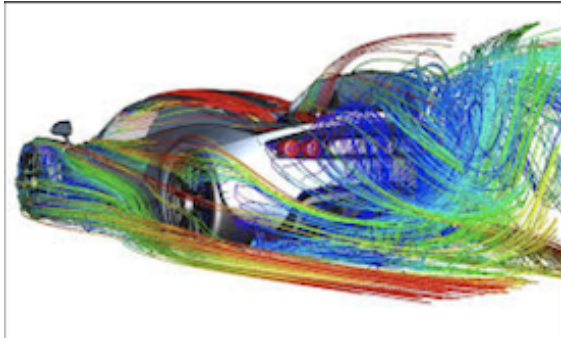
(b) Análisis sísmico en sistemas Multi-GPU, para mejorar interpretación de datos en la exploración de gas y petróleo



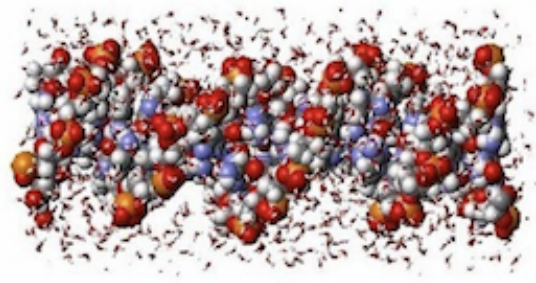
(a) Diseño realistico en sistemas Muti-GPU, acorta tiempos de diseño logrando llegar más rápido al mercado.



(b) Procesamiento de imágenes médicas, para manipular imágenes en 3D

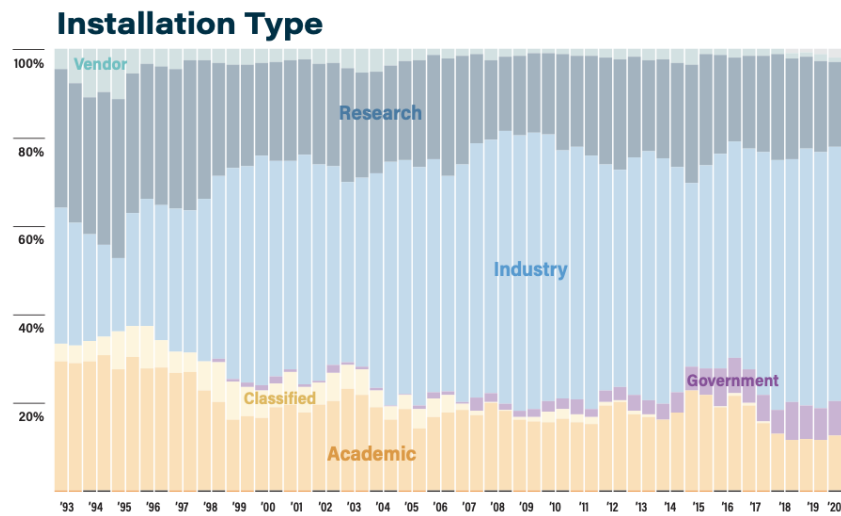


(a) Dinámica de flúidos, utiliza software sofisticado que requiere alto poder computacional



(b) Dinámica molecular, para mejorar productos que no afecten el medio ambiente

Avance industrial/comercial/académico en supercomputación



Fuente: <https://www.top500.org>

Referencias

- [1] The Lives of a Cell, Notes of a biology watcher, Lewis Thomas, 1971-73
- [2] Michigan Today, February 1991, Vol. 23, No.1 *There is a supercomputing race between - The Chickens and the Ox*
- [3] Computer Architecture, A Quantitative Approach, Hennesy & Patterson, 4th edition, 2007. Chap.5
- [4] Top 500 Supercomputación: <https://www.top500.org>
- [5] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.